

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
uploaded = files.upload()

image_path = list(uploaded.keys())[0]

print("Image uploaded:", image_path)
```

wolverine.webp

wolverine.webp(image/webp) - 221374 bytes, last modified: 9/22/2026 - 100% done
Saving wolverine.webp to wolverine.webp
Image uploaded: wolverine.webp

```
image = cv2.imread(image_path)

if image is None:
    print("Error: Image could not be loaded.")
else:
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    print("Image loaded successfully!")
    print("Image dimensions:", gray.shape)
    print("Data type:", gray.dtype)
```

Image loaded successfully!
Image dimensions: (779, 960)
Data type: uint8

```
plt.figure(figsize=(8, 6))

plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.show()
```

Original Grayscale Image



```
# Convert image to floating point
image_float = np.float32(gray)

# Calculate 2D Fourier Transform
dft = np.fft.fft2(image_float)

print("Discrete Fourier Transform calculated successfully!")
print("DFT shape:", dft.shape)
```

```
Discrete Fourier Transform calculated successfully!
DFT shape: (779, 960)
```

```
# Shift zero-frequency component to the center
dft_shift = np.fft.fftshift(dft)

print("Zero-frequency component shifted to the center.")
```

```
Zero-frequency component shifted to the center.
```

```
# Calculate magnitude spectrum
magnitude_spectrum = np.abs(dft_shift)

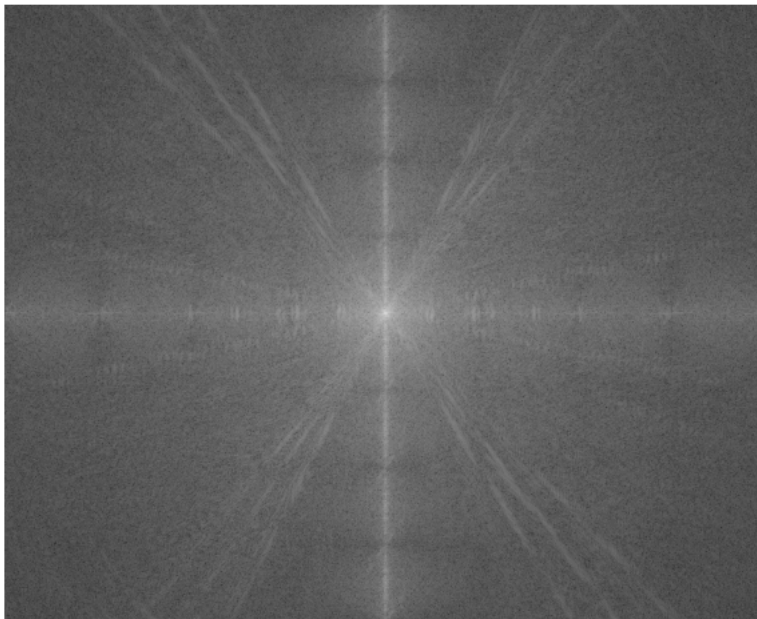
# Use logarithmic scale for better visualization
magnitude_spectrum_log = np.log1p(magnitude_spectrum)

plt.figure(figsize=(10, 6))

plt.imshow(magnitude_spectrum_log, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.show()
```

Magnitude Spectrum



```
rows, cols = gray.shape

# Find center of the frequency spectrum
crow, ccol = rows // 2, cols // 2

# Create a mask filled with zeros
low_pass_mask = np.zeros((rows, cols), np.uint8)

# Radius of the low-pass filter
radius = 40

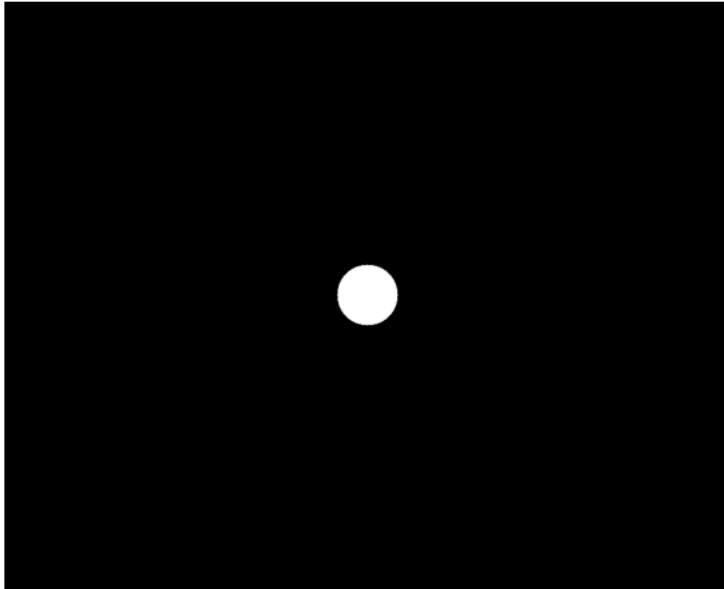
# Create circular low-pass filter
cv2.circle(
    low_pass_mask,
    (ccol, crow),
    radius,
    1,
    -1
)
```

```
plt.figure(figsize=(7, 7))

plt.imshow(low_pass_mask, cmap="gray")
plt.title("Low-Pass Filter Mask")
plt.axis("off")

plt.show()
```

Low-Pass Filter Mask



```
# Apply mask to frequency spectrum
low_pass_dft = dft_shift * low_pass_mask

# Shift frequency spectrum back
low_pass_ishift = np.fft.ifftshift(low_pass_dft)

# Perform inverse Fourier Transform
low_pass_image = np.fft.ifft2(low_pass_ishift)

# Get magnitude and convert to uint8
low_pass_image = np.abs(low_pass_image)
low_pass_image = np.uint8(
    np.clip(low_pass_image, 0, 255)
)

print("Low-pass filtering completed.")
```

Low-pass filtering completed.

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(low_pass_image, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Low-Pass Filtered Image



```
# Create a high-pass filter
high_pass_mask = np.ones((rows, cols), np.uint8)

# Remove the low-frequency center
cv2.circle(
    high_pass_mask,
    (ccol, crow),
    radius,
    0,
    -1
)

plt.figure(figsize=(7, 7))

plt.imshow(high_pass_mask, cmap="gray")
plt.title("High-Pass Filter Mask")
plt.axis("off")

plt.show()
```

High-Pass Filter Mask



```
# Apply high-pass mask
high_pass_dft = dft_shift * high_pass_mask

# Shift back
high_pass_ishift = np.fft.ifftshift(high_pass_dft)
```

```
# Inverse Fourier Transform
high_pass_image = np.fft.ifft2(high_pass_ishift)

# Magnitude
high_pass_image = np.abs(high_pass_image)

# Normalize for display
high_pass_image = cv2.normalize(
    high_pass_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

print("High-pass filtering completed.")
```

High-pass filtering completed.

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(high_pass_image, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(16, 10))

plt.subplot(2, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum_log, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(low_pass_image, cmap="gray")
plt.title("Low-Pass Filtered")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(high_pass_image, cmap="gray")
```

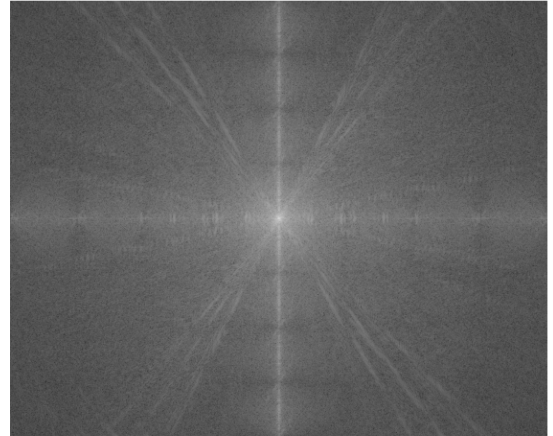
```
plt.title("High-Pass Filtered")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

Original Image



Magnitude Spectrum



Low-Pass Filtered



High-Pass Filtered



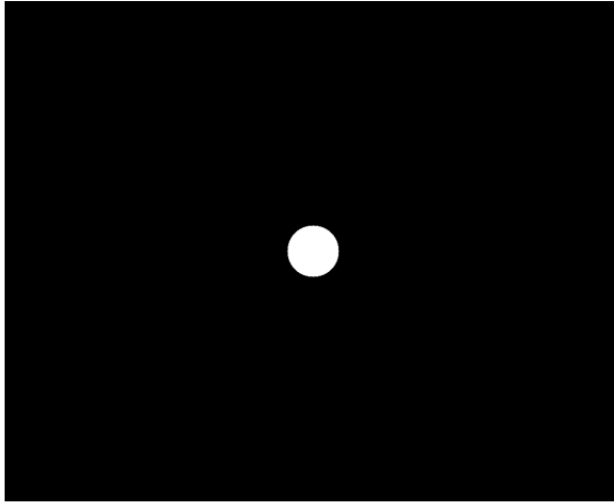
```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.imshow(low_pass_mask, cmap="gray")
plt.title("Low-Pass Filter Mask")
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
plt.imshow(high_pass_mask, cmap="gray")
plt.title("High-Pass Filter Mask")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

Low-Pass Filter Mask



High-Pass Filter Mask



OBSERVATIONS:

1. The Fourier Transform converted the image from the spatial domain into the frequency domain.
2. The magnitude spectrum showed the distribution of frequency components in the image.
3. The center of the shifted spectrum represented low-frequency components.
4. The Low-Pass Filter removed high-frequency components and produced a smoother image.
5. The Low-Pass Filter reduced fine details and some noise.
6. The High-Pass Filter removed low-frequency components and highlighted edges and fine details.
7. Frequency domain filtering provides a useful way to selectively manipulate different frequency components of an image.

Q1. What is the Fourier Transform? Why is it important in digital image processing?

Answer: The Fourier Transform is a mathematical technique that converts an image from the spatial domain into the frequency domain. It represents an image using different frequency components. It is important because it allows us to analyze and modify low and high frequency components separately. It is widely used for image filtering, enhancement, restoration, and noise removal.

Q2. Differentiate between the spatial domain and the frequency domain.

Answer: The spatial domain represents an image directly using its pixels. Operations are performed directly on pixel values.

The frequency domain represents an image using its frequency components after applying a Fourier Transform.

Spatial domain processing works directly on pixels, while frequency domain processing modifies different frequency components of an image.

Q3. What is the significance of the Discrete Fourier Transform (DFT) in image processing?

Answer: The Discrete Fourier Transform converts a digital image from the spatial domain into the frequency domain. It helps identify low-frequency and high-frequency components and allows frequency-based filtering to be performed. DFT is useful for image enhancement, noise removal, restoration, compression, and feature analysis.

Q4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

Answer: Normally, the zero-frequency component is located at the corner of the Fourier spectrum. The `fftshift` operation moves the zero-frequency component to the center. This makes the frequency spectrum easier to visualize and makes it easier to design circular low-pass and high-pass filters around the center.

Q5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

Answer: A Low-Pass Filter allows low-frequency components to pass and reduces high-frequency components. It produces a smoother image and can be used for noise reduction and image smoothing.

A High-Pass Filter allows high-frequency components to pass and reduces low-frequency components. It emphasizes edges and fine details and can be used for edge enhancement and feature extraction.

Q6. What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?

Answer: The Inverse Fourier Transform converts the processed frequency-domain data back into the spatial domain. After applying a frequency filter, the IDFT reconstructs the filtered image so that it can be displayed and analyzed.

Q7. Why is frequency domain filtering preferred for certain image enhancement tasks?

Answer: Frequency domain filtering allows specific frequency components to be selected, removed, or enhanced. It is useful when we need to control low-frequency and high-frequency information separately. It can be particularly effective for noise removal, smoothing, sharpening, periodic noise reduction, and image restoration.

Q8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

Answer: Four applications are:

1. Medical image analysis
2. Satellite and remote sensing image processing
3. Image restoration and noise removal
4. Biometric and pattern recognition systems

Q9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Answer: Spatial domain filtering operates directly on image pixels and is simple and suitable for operations such as blurring, sharpening, thresholding, and edge detection.

Frequency domain filtering first converts the image into frequency components and then applies filters to those components.

For large images or certain complex filtering operations, frequency domain methods can be computationally efficient, especially when Fast Fourier Transform (FFT) algorithms are used.